

Ring of the Middle Earth - Architecture Document

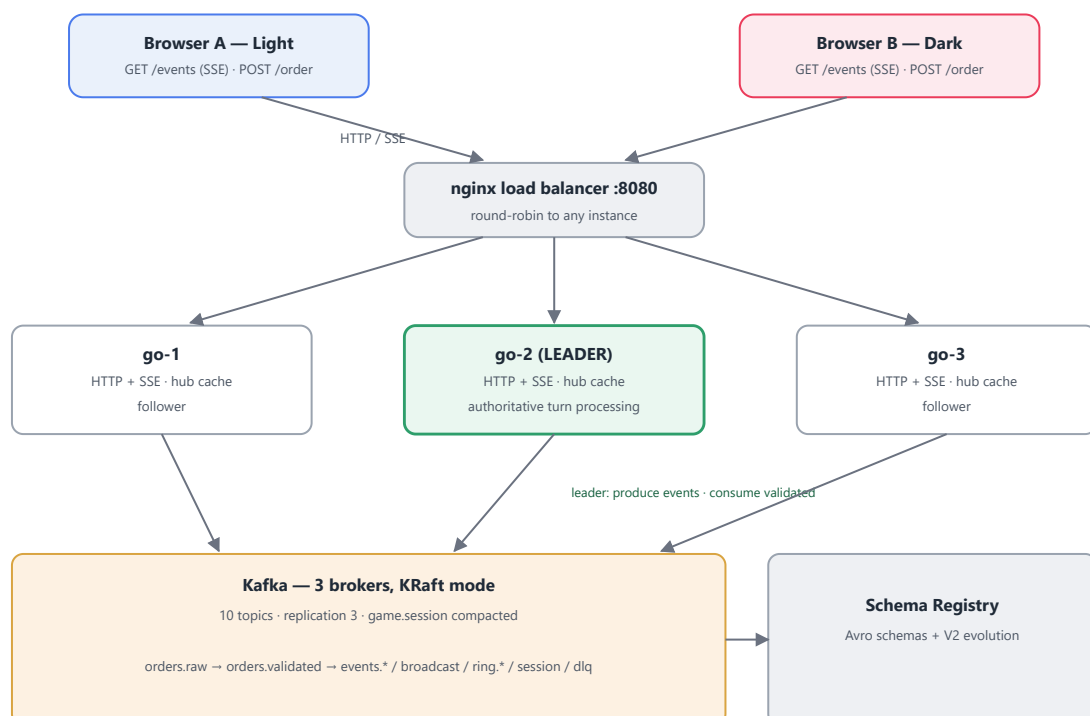
Distributed Application Development term project · Option B (Go) · Human-vs-Human strategy game on a Kafka event backbone.

1. Technology choice and overview

We built the system in **Go** (Option B). The game engine, the order-validation and route-risk services, the HTTP/SSE layer and the browser client are all Go plus vanilla JavaScript. A pure-Go Kafka client (`twmb/franz-go`) is used so the build needs no C toolchain, and Avro records are encoded with `hamba/avro` in the Confluent wire format (a magic byte, a 4-byte schema id, then the Avro body). The messaging tier - three Kafka brokers in KRaft mode, the Confluent Schema Registry and a Kafka web UI - runs in Docker.

The application tier is three interchangeable engine instances (`go-1`, `go-2`, `go-3`) behind an nginx load balancer. Every instance can answer reads and accept orders. Exactly one instance is elected leader and runs the authoritative turn processing. All durable state lives in Kafka, so any instance can be killed and the survivors carry on. This is the central idea of the Go option: *the application tier is stateless, the broker holds the state*.

2. System diagram

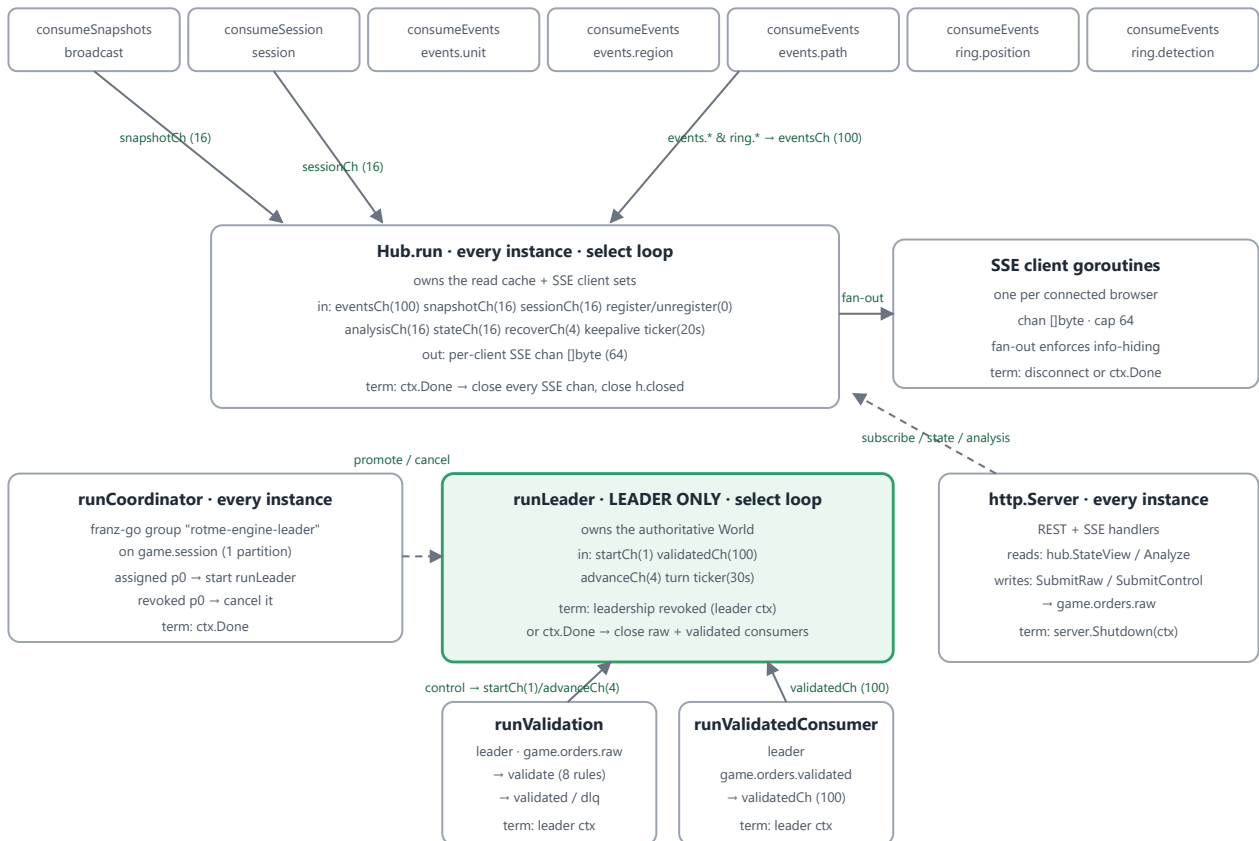


Reads (`/game/state`, `/orders/available`, `/analysis/*`, `/events`) are served by any instance from a local cache that is rebuilt from Kafka. Order submission and the two control actions (start game, advance turn) are published to `game.orders.raw`; only the leader consumes and acts on them.

3. Goroutine map (Option B)

State is never shared by locking it; each long-lived goroutine owns its state and is reached through channels. Channel buffer capacities are shown in parentheses; every box lists its termination condition.

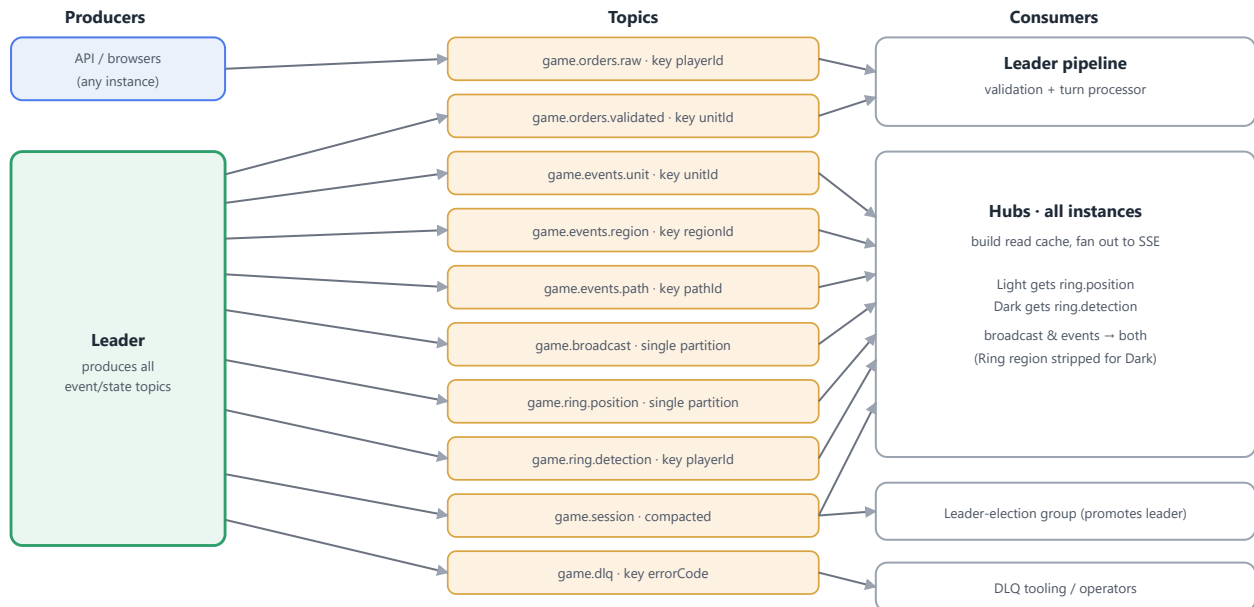
Kafka consumer goroutines — one per subscribed topic, on every instance · term: `ctx.Done`



The leader is the only writer of the `World`. The validation goroutine never touches the `World`; it reads an immutable `ValidationState` snapshot the leader republishes after every turn (an `atomic.Pointer`), which removes the data race entirely. When an instance loses leadership its leader context is cancelled so the validation and turn-consumer goroutines return; when the process exits, the hub closes every SSE channel. Nothing is left running, which the goroutine count confirms after a game.

4. Kafka topics

The diagram shows the flow through the ten topics; the table below gives the exact producer, consumer and settings for each.



Topic	Key	Part.	Repl.	Cleanup	Produced by	Consumed by
game.orders.raw	playerId	3	3	delete 1h	API (order intake, control)	leader validation
game.orders.validated	unitId	6	3	delete 1h	leader validation + enrichment	leader turn processor
game.events.unit	unitId	6	3	delete 7d	leader	all hubs (SSE/cache)
game.events.region	regionId	6	3	delete 7d	leader	all hubs
game.events.path	pathId	6	3	delete 7d	leader	all hubs
game.session	-	1	3	compact	leader	leader-election group
game.broadcast	-	1	3	delete 1h	leader (snapshot + GameOver txn)	all hubs
game.ring.position	-	1	3	delete 1h	leader	Light SSE only
game.ring.detection	playerId	2	3	delete 1h	leader	Dark SSE only
game.dlq	errorCode	3	3	delete 7d	leader validation	operators

Partition-key rationale. Orders are keyed by `playerId` so one player's submissions keep their relative order. Validated orders and unit events are keyed by `unitId` so everything about a single unit lands on one partition in order. Region and path events are keyed by their entity id for the same reason. The single-partition topics (`game.session`, `game.broadcast`, `game.ring.position`) carry global, totally-ordered state where partitioning would only create races. `game.session` is log-compacted so it always retains the latest session record, which is what the leader-election group consumes.

Information hiding. The Ring Bearer's true position only ever travels on `game.ring.position`, which the hub forwards to Light clients only. `game.broadcast` snapshots carry the Ring Bearer's region as the empty string. So the Dark browser physically never receives the position over any channel.

Schema evolution & exactly-once. `OrderValidated` is registered V1, then a backward-compatible V2 adds a nullable `routeRiskScore` (default null) - V1 consumers keep working. The `GameOver` event is produced inside a Kafka transaction with an idempotent producer, so it appears on `game.broadcast` exactly once even if the leader crashes mid-publish (the transaction either commits once or aborts).

5. Order validation and route-risk (the two topologies)

Topology 1 (validation) and Topology 2 (route-risk enrichment) are implemented as ordinary Go consumer logic rather than the JVM Kafka Streams library, which keeps the whole project in one language. The leader consumes `game.orders.raw`, validates each order against its in-memory state (which is exactly the `KTable` the spec describes), and writes the result to `game.orders.validated` or, on failure, `game.d1q` with the matching error code. Route orders are then enriched: the leader computes the risk score (region threat + surveillance·3 + blocked·5 + threatened·2 + nazgul-proximity·2) and re-emits the record with `routeRiskScore` set.

6. Paradigm justification

Why Go fits this problem

The game is a stream of small, independent messages - orders in, events out - with a single authoritative reducer (the turn). Go's goroutines-and-channels model maps onto this directly: a consumer goroutine per topic, an owner goroutine per piece of state, and channels as the only way in. Making the application tier stateless and pushing all durable state into Kafka means fault tolerance is mostly free: a dead instance is just a consumer-group member that left, and Kafka's rebalance protocol does the rest. We did not have to build our own clustering or replication.

What was genuinely harder than with actors

Single-writer ownership of the world. Akka gives you a cluster singleton out of the box; in Go we had to build leader election ourselves (a single-partition consumer group) and make sure only the leader runs turn processing. Recovering an in-progress game after a failover is also manual: there is

no actor with persisted state to rehydrate, so every turn the leader writes the full world to the compacted `game.session` topic, and a promoted leader rebuilds from the latest such record. The subtle part is that this record has to carry the Ring Bearer's route, which is secret, so it goes on `game.session` (never forwarded to a browser) rather than on the broadcast snapshot the Dark side reads.

How actors would solve the two hardest parts

Single-writer state: Akka's `RingBearerActor` / `WorldStateActor` cluster singletons are the leader by construction, so the election we hand-rolled disappears. Recovery: Akka Persistence event-sources each actor, so a crashed singleton restarts and replays its own journal to the exact pre-crash state, routes included - no snapshot-shaped data loss. The cost is the opposite of ours: state is bound to the actor and the framework, the cluster has to be configured and reasoned about, and the JVM footprint is larger.

7. Reflection

The part that took the most rethinking was the boundary between "state lives in the app" and "state lives in Kafka". Early on the engine simply held the world in memory and that was the truth; everything was easy and the tests flew. The moment we introduced three instances, that assumption broke. Two questions appeared that had not existed before: who is allowed to mutate the world, and where does a newly-promoted instance get its starting point? The first we answered with leader election over a single-partition topic, which is a small amount of code but a large amount of careful thinking about the assignment and revocation callbacks running on a different goroutine than the leader loop. The second we answered by having every instance keep a read-only world cache rebuilt from the broadcast topic, so a promoted leader already has something to adopt.

What surprised us was how much the information-hiding requirement shaped the topic design rather than the code. It is tempting to strip the Ring Bearer's position in a handler somewhere, but the robust answer is to never let it travel on a channel the Dark side reads: the position has its own topic, and the shared snapshot carries an empty string. Once the topics were right, the code that enforces the rule became a one-line switch, and the race test that proves the Dark view is always empty passed without drama. We came to see information hiding as a data-flow property, not a code property.

If we designed it again we would separate the read cache and the leader's authoritative world into clearly different types from the start, instead of reusing the same struct, because conflating them cost us an afternoon of confusion about which copy was the source of truth. We would also decide up front which state is secret and which is public, because that one distinction is what determines whether a piece of data may travel on the broadcast topic or has to ride the private session topic; we learned it by first putting the Ring's route on the wrong channel. The biggest lesson was that in a Kafka-centred design, most of the hard problems are decided by where a piece of data is keyed and which topic it flows on; the Go code that consumes it is almost an afterthought once those choices are right.

Appendix - How to run

```
# from the repo root (Windows + Docker Desktop)
powershell -File scripts/up.ps1          # or: make up    -> builds + starts everything
#   Light browser: http://localhost:8080/?side=light
#   Dark  browser: http://localhost:8080/?side=dark
#   Kafka UI:      http://localhost:8088

powershell -File scripts/test.ps1        # or: make test    (unit tests, no Docker)
powershell -File scripts/test-race.ps1   # or: make test-race (router race tests in a Linux container)

# fault tolerance demo
docker stop go-2          # if go-2 is the leader, another instance is promoted and recovers from Kafka
docker start go-2         # rejoins the consumer group
```